

1. 개요

Apriori 알고리즘을 통해 frequent pattern을 찾고 association rule을 찾아 각각의 support와 confidence를 출력한다.

2. 알고리즘 요약

1) Downward Closure Property

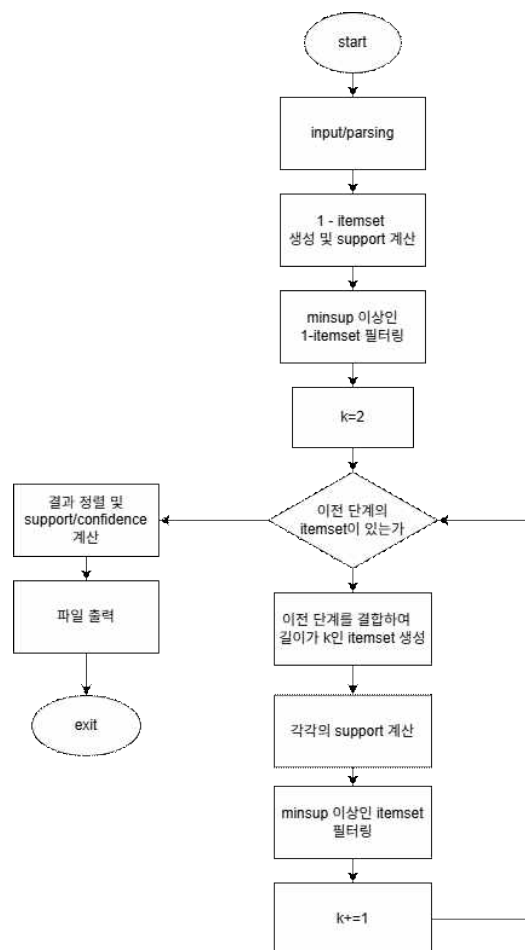
어떤 itemset이 frequent하다면, 그 모든 subset들이 무조건 frequent하다.

2) Apriori Pruning Principle

Downward Closure Property의 대우로 frequent 하지 않은 itemset이 있다면 그 superset은 frequent 하지 않다.

3) flow chart

Apriori Pruning Principle에 따라 프로그램 전체의 흐름을 그려보면 다음과 같다.



<그림 1> 프로그램 전체 흐름

우선 input 파일을 읽어 각 줄을 parsing 해 각 transaction을 저장한다. 그 이후 1-itemset을 생성해 support count를 계산한다. 그 비율이 minimum support 이상인 itemset만 필터링해 frequent itemset에 보관한다.

이후 길이가 k=2부터 시작해서 하나씩 늘어나는 itemset을 self-joining을 통해 만들고 그 support가 minimum support 이상인 itemset을 필터링해 frequent itemset에 추가한다.

다음 단계를 진행 전에 만약 길이가 k인 frequent itemset이 없다면, 더 긴 frequent itemset은 검사하지 않아도 되므로 출력단계로 넘어간다.

출력단계에서는 길이가 2 이상인 frequent itemset을 둘로 쪼개 각각의 support와 confidence를 계산해 출력한다.

본 과제에서는 minimum confidence나 Lift에 관한 조건이 없었으므로 전체를 출력한다.

3. 코드 설명

1) minimum support 및 입출력 파일 위치

```
minSup = float(sys.argv[1])
inputFile = sys.argv[2]
outputFile = sys.argv[3]
```

<그림 2> min support 및 입출력 파일 위치
인자로 전달된 minimum support와 입출력 파일의 위치를 저장한다.

2) file read 및 transaction 저장

```
transactions = []

with open(inputFile, 'r') as file:
    for line in file:
        items = line.strip().split('\t')
        transaction = set(map(int, items))
        transactions.append(transaction)
```

<그림 3> file read 및 transaction 저장

먼저 파일을 한 줄씩 읽어 transaction을 만든다. 명세에 한 transaction에 중복된 요소가 없다고 되어 있지만 중복을 피하기위해 set으로 저장했다. 이후 이 transaction을 list에 차례대로 저장한다. transaction은 이후 support_count를 계산하는데 사용한다.

3) 1-itemset 생성 및 필터링

```
counts = {}
frequentItemsets = {}
tmpList = []

for transaction in transactions:
    for item in transaction:
        itemset = frozenset([item])
        counts[itemset] = counts.get(itemset, 0) + 1

total = len(transactions)
minCount = minSup * total / 100
frequentItemsets = {key: value for key, value in counts.items() if value >= minCount}
tmpList = list(frequentItemsets.keys())
```

<그림 4> 1-itemset 생성 및 필터링

counts는 현재 단계에서 frequent itemset에 들어갈 수 있는 후보군을 담아두는 dictionary이다. 또한 frequentItemsets는 도출된 모든 frequent Itemset을 저장하는 dictionary이다.

둘 모두 itemset을 key로 하여 각각의 support를 저장하기 위해 dictionary로 저장했다. tmpList는 현재 단계에서 만들어진 frequent Itemset을 저장하기 위한 리스트이다. 이는 다음 단계의 길이가 k인 후보 Itemset을 만들 때, 길이가 k-1인 frequent Itemset의 self-join으로 만들기 때문이다.

우선 모든 transaction을 순회하며, 각각의 길이가 1인 item이 몇 번 나왔는지 count 한다. 이때 key를 frozen set으로 변환한 이유는 이후에는 counts의 key가 itemset이 될 것인데, 이때 set을 key로 사용이 불가능하기 때문이다.

이후 처음에 입력된 minimum support를 토대로 support count가 minCount이상인 itemset만 필터링 해 frequentItemsets에 넣고, 그 key값(길이가 1인 itemset)은 tmpList에 넣는다.

4) 길이가 k인 frequent itemset 생성

```
k = 2
while(len(tmpList) > 0):
    counts = {}
    candidates = set()
    tmpSet = set(tmpList)

    for i in range(len(tmpList)):
        for j in range(i+1, len(tmpList)):
            candidate = tmpList[i].union(tmpList[j])
            if len(candidate) == k:
                isValid = True
                for item in candidate:
                    if candidate - frozenset([item]) not in tmpSet:
                        isValid = False
                        break
                if isValid:
                    candidates.add(candidate)

    for transaction in transactions:
        for candidate in candidates:
            if candidate.issubset(transaction):
                counts[candidate] = counts.get(candidate, 0) + 1

    frequentCandidate = {key: value for key, value in counts.items() if value >= minCount}
    frequentItemsets.update(frequentCandidate)
    tmpList = list(frequentCandidate.keys())
    k += 1
```

k=2부터 시작하여 길이가 1씩 늘어나는 frequent Itemset을 찾는 과정이다. 우선 이전단계의 frequent Itemset인 tmpList가 비어있지 않은 경우에만 다음단계로 넘어간다.

우선 self-joining을 통해 가능한 후보 candidate를 생성한다. 코드에서는 이전 단계에서의 모든 frequent Itemset을 각각 union해 길이가 k인 경우에만 후보 candidate에 포함되도록 구현했다.

또한 Apriori Pruning Principle에 따라 어떤 subset이라도 frequent하지 않다면 이 candidate또한 frequent하지 않기 때문에, 길이가 k-1인 모든 subset이 tmpSet (직전단계에서 만들어진 frequentItemset)에 포함되어 있는지 확인해 아닌 경우 후보에서 제외하는 pruning을 진행한다.

이후에는 길이가 1일 때와 마찬가지로 transaction을 순회하며 해당 candidate가 포함되었는지 확인해 support count를 계산하고, support count가 minimum support count이상인 경우에만 frequent Itemset에 추가하고, tmpList를 길이가 k인 frequent Itemset으로 초기화해 길이가 k+1인 frequent Itemset을 찾는데 사용한다.

직전 단계의 frequent Itemset이 없으면 반복문을 종료한다.

4) 정렬 및 출력

```
sortedItems = sorted(frequentItemsets.items(), key=lambda x: (len(x[0]), tuple(sorted(x[0]))))
with open(outputFile, 'w') as out:
    for itemset, supportCount in sortedItems:
        n = len(itemset)
        if n < 2:
            continue

        itemsetList = sorted(list(itemset))

        for i in range(1, (1 << n) - 1):
            XList = []
            for j in range(n):
                if i & (1 << j):
                    XList.append(itemsetList[j])

            X = frozenset(XList)
            Y = itemset - X

            support = (supportCount / total) * 100
            confidence = (supportCount / frequentItemsets[X]) * 100

            support = support + 1e-6
            confidence = confidence + 1e-6

            strX = "{" + ",".join(map(str, sorted(X))) + "}"
            strY = "{" + ",".join(map(str, sorted(Y))) + "}"

            out.writelines(f"{strX}\t{strY}\t{support:.2f}\t{confidence:.2f}\n")
```

<그림 6> 정렬 및 출력

명세에는 출력 순서는 중요하지 않다고 되어 있지만 최대한 제공된 output과 동일하게 맞추기 위해 시도했다. 우선 itemset의 길이와 item의 순서를 기준으로 정렬해 sortedItems라는 리스트에 itemset과 count의 tuple을 저장한다.

길이가 1인 경우에는 association rule을 만들지 못하므로 2이상인 경우에 대해서만 고려한다. 출력 예시에서 길이가 3개 이상인 경우에 아래와 같이 표현되어 있는데, 이는 정렬된 itemset을 비트마스크를 통해 association rule을 만든 순서와 동일함을 알아채 코드와 같이 구현했다. 또한 support와 confidence도 계산하는데 confidence의 경우 조건부확률이므로 X 가 등장했을 때 $(X \cup Y)$ 가 등장할 확률로 계산했으며, 이후 출력형식에 맞게 출력했다.

명세에서 소수점 둘째 자리까지 반올림하라고 하여 1e-6를 더해 반올림하여 출력했다.

{1}	{5,8}	5.40	18.12
{5}	{1,8}	5.40	21.43
{1,5}	{8}	5.40	54.00
{8}	{1,5}	5.40	11.95
{1,8}	{5}	5.40	35.06
{5,8}	{1}	5.40	42.86

<그림 7> 출력 예시

4. 실행

1) 실행환경

OS : Window, Python Version : Python 3.12.2

2) 실행 방법

```
python apriori.py 5 input.txt output.txt
```

<그림 8> 실행 방법

apriori.py파일이 있는 경로에서 python apriori.py [min sup] [입력파일] [출력파일] 형식으로 실행하시면 됩니다. 기본적으로는 apriori.py와 입력파일이 같은 경로에 있는 것을 가정합니다.

3) 실행 결과

Itemset	Support	Confidence
{1}	5.40	18.12
{5}	5.40	21.43
{1,5}	5.40	54.00
{8}	5.40	11.95
{1,8}	5.40	35.06
{5,8}	5.40	42.86

<그림 9> 실행 결과

실행 결과 제공된 파일과 association rule이 동일한 순서의 output.txt 파일을 생성했다.